Java™

Contents

El Alquimista Supremo de la Dimensión Pixel 3

I. Prólogo 4

II. Introducción 5

V. Ejercicio 0: El Registro de Héroes (record & ArrayList) 6

VI. Ejercicio 1: Procesador de Loot (Filtro de Duplicados) 7

VII. Ejercicio 2: El Mapa de la Ciudad (Map & HashMap) 8

VIII. Ejercicio 3: La Cola de la Mazmorra (Queue & Deque) 9

IX. Ejercicio 4: La Pila de Hechizos (Stack) 11

X. Ejercicio 5: El Alquimista de Datos 12

X. Ejercicio 6: Misión de Comandos (ArrayList & String Processing) 13

X. Ejercicio 7: El Escáner de Terreno (Arrays Estáticos) 14

XIII. Ejecución 15

El Alquimista Supremo de la Dimensión Pixel

Los datos en la Dimensión Pixel se han vuelto inestables. Como Alquimista Supremo, ya no solo guardas objetos; debes transmutar flujos de información, gestionar identidades inmutables y optimizar la memoria del Reino utilizando el ***Java Collections Framework*** y las últimas funciones de Java 17/21.

Instrucciones Generales

- ✓ ***Versión:*** Java 17+ (OpenJDK).
- ✓ ***Estructura:*** Una clase por archivo (Puedes usar package).
- ✓ ***Linter Obligatorio:*** Checkstyle (Google Java Style). Errores de estilo = KO.

- ✓ ***Comentarios:*** Obligatorios, solo en inglés y siguiendo formato Javadoc.
- ✓ **Nomenclatura:** Clases en PascalCase, métodos y variables en camelCase.
- ✓ ***IDE:*** IntelliJ IDEA (recomendado), VS Code, Eclipse, etc.
- ✓ ***Compilación:*** Vía terminal: javac Clase.java y java Clase.

Entrega y Evaluación

- Entrega en repositorio Git.

I. Prólogo

El Tejido de la Realidad Digital

En los albores de la Dimensión Píxel, los antiguos escribas usaban listas simples para contar flechas y pociones. Pero el Reino ha crecido. Ahora, miles de aventureros cruzan portales cada segundo, dejando tras de sí un rastro de metadatos, logros y hechizos superpuestos.

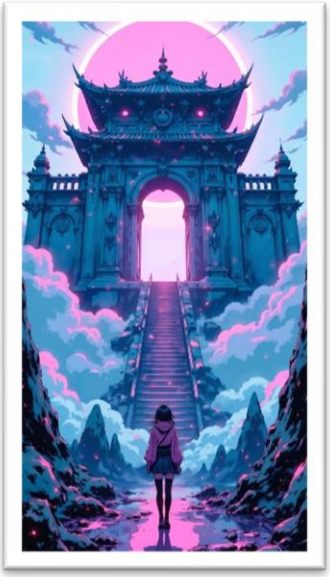
Como Alquimista Supremo, tu grimorio es el **Collections Framework**. Has comprendido que la realidad no es una masa informe de datos, sino una estructura que debe ser elegida con precisión quirúrgica. Un segundo de retraso en la búsqueda de una coordenada puede significar la pérdida de una expedición entera.

Hoy, aprenderás a elegir el recipiente perfecto para cada esencia digital: desde la velocidad del rayo de un HashMap hasta el orden inquebrantable de un TreeSet.

II. Introducción

¡Bienvenido a **Data Quest: ¡El Alquimista Supremo!**

Tras haber dominado el arte de la resiliencia y el manejo de desastres en el módulo anterior, ahora te enfrentas a un reto de escala masiva. En la **Dimensión Pixel**, los datos ya no son goteos aislados de información; son torrentes que fluyen desde miles de fuentes simultáneas. Tu éxito como ingeniero ya no depende solo de que el código "no se rompa", sino de qué tan eficiente y elegante es tu forma de organizar la realidad.



En este módulo, dejarás atrás las variables simples para dominar el **Java Collections Framework**, el corazón latente de cualquier aplicación profesional en Java. Aprenderás que programar es, en esencia, tomar decisiones sobre estructuras:

- ✓ ***¿Es este dato único?*** Usarás un **set.**
- ✓ ***¿Necesitas encontrar un valor entre un millón en un parpadeo?*** Usarás un-Map.
- ✓ ***¿Debes mantener el orden exacto de una crónica?*** Usarás una **List**.

Además, introduciremos los **Records**, una de las funciones más potentes de las versiones modernas de Java, que te permitirán modelar a tus héroes y objetos con una claridad y seguridad que los antiguos programadores solo podían soñar.
Prepárate, Alquimista. Estás a punto de aprender a transmutar el caos de la información pura en sistemas de datos perfectamente estructurados y ultra-eficientes. La Dimensión Pixel espera tus órdenes.

¿Qué dominarás en esta etapa?

1. ***Modelado Inmutable:*** Uso de record para crear contenedores de datos indestructibles.
2. ***Gestión Dinámica:*** Diferencia práctica entre el almacenamiento estático (Array) y el dinámico (ArrayList).
3. ***Algoritmos de Búsqueda:*** Implementación de diccionarios con HashMap para acceso instantáneo.
4. ***Flujos de Trabajo:*** Control de colas y pilas para procesos lógicos complejos (Deque y Stack).
5. **Pureza de Datos:** Eliminación de duplicados y ordenación automática mediante Sets.

V. Ejercicio 0: El Registro de Héroes (record & ArrayList)

- **Directorio:** ex0/
- **Archivo:** HeroRegistry.java, Adventurer.java
- **Funciones Autorizadas**: ArrayList<Adventurer>, .add(new Adventurer(...)), for-each, size().
- **Método**: public record Adventurer(String name, int level, String role) {}, public static void main(String[] args).
- **Restricción:**

  ✓ **No uses class** para el aventurero. El record es obligatorio.
  ✓ ***Un record no tiene "setters".*** Si quieres cambiar el nivel, tienes que crear un objeto nuevo.

Objetivo - Crear una lista de aventureros inmutable.

- Usa un record**.**
- Almacena varios registros en un ***ArrayList***.
- Salida: Imprime la lista completa y el tamaño de la expedición.

Salida Esperada

```
=== ADVENTURER REGISTRY ===
- Artorias (Level 50) [Knight]
- Sylvanas (Level 45) [Archer]

Total: 2 heroes in the party.
```

VI. Ejercicio 1: Procesador de Loot (Filtro de Duplicados)

- **Directorio:** ex1/
- **Archivo:** LootProcessor.java
- **Método:** public static void main(String[] args).
- **Funciones Autorizadas:** HashSet<String>, TreeSet<String>, List<String>, addAll(miLista), System.out.println(), add, size.
- **Restricción:**
  - ✓ **No uses bucles if** para ver si un item ya está en la lista. Deja que el ***Set*** haga el trabajo sucio (esa es su magia).
  - ✓ **No uses Collections.sort().** Debes usar un TreeSet para que el orden sea automático por estructura.

Objetivo - Eliminar duplicados de un botín de guerra.

- Recibe una lista de items (ej: "Sword", "Shield", "Sword", "Potion").
- Usa un HashSet para filtrar y quedarte solo con los items únicos.
- Usa un TreeSet para que, además de únicos, salgan ordenados alfabéticamente.

Salida Esperada

```
=== LOOT PROCESSING ===
Raw Loot: [Sword, Shield, Sword, Potion]
Unique Loot: [Sword, Shield, Potion]
Sorted Loot: [Potion, Shield, Sword]

Total unique items: 3
```

VII. Ejercicio 2: El Mapa de la Ciudad (Map & HashMap)

- **Directorio:** ex2/
- **Archivo:** CityMap.java
- **Método**: public static void searchLocation(HashMap<String, String> map, String place), public static void main...
- **Funciones Autorizadas:** HashMap<String, String>, put(key, value), .get(key), .containsKey(key)
- **Restricción:**
  - ✓ ***No uses listas para buscar*** (nada de recorrer un ***ArrayList*** con un if). El Map debe encontrar el lugar directamente por su llave.
  - ✓ **Tipado:** El mapa debe estar definido claramente como ***<String, String>*.**

Objetivo - Asignar coordenadas a localizaciones.

- Usa un ***HashMap<String, String>*** donde la clave sea el nombre del lugar y el valor la coordenada.

Key	Value
Inn	Located at the North Plaza.
Guild	Located at the Town Center.
Blacksmith	Located near the East Gate.
Market	Located in the South District.

Salida esperada

```
=== PIXEL CITY GPS ===
Registering locations...
Search: 'Inn' -> Found: Located at the North Plaza
Search: 'Graveyard' -> Error: This location is not
discovered yet.

Total locations in map: 4
```

VIII. Ejercicio 3: La Cola de la Mazmorra (Queue & Deque)

- **Directorio:** ex3/
- **Archivo:** DungeonQueue.java
- **Método**: public static void main...
- **Funciones Autorizadas:** ArrayDeque<String>, ArrayDeque<String>, addLast(name), addFirst(name), pollFirst(), peekFirst(), isEmpty()

- **Restricción:**
  - ✓ **No uses Stack** para este ejercicio; necesitamos comportamiento de cola.
  - ✓ **No uses un ArrayList** para simular la cola (eliminar el primer elemento de un ArrayList es muy lento e ineficiente).

Objetivo - Simular un sistema de entrada a una mazmorra donde:

- Se usa comportamiento de **cola (Queue → FIFO)**
- Se usa comportamiento de **doble cola (Deque)**
- Todo implementado con **ArrayDeque**

Datos: "Warrior", "Mage", "Rogue", "Healer", "Assassin", "Boss".
Debes saber cómo manejar estos datos y saber cuándo ponerlos al inicio, al final, cuando agregarlo o eliminarlo, eso ya depende de ti y la salida que te muestren.

Tipo	Qué es	Características	Para qué se usa
`Queue`	Interfaz	FIFO (primero en entrar, primero en salir)	Colas normales: turnos, procesos, BFS
`Deque`	Interfaz	Doble extremo (inicio y final)	Colas + pilas (más flexible)
`ArrayDeque`	Clase (implementa `Deque`)	Rápida, eficiente, sin `null`	Implementación real de `Queue` y `Deque`

Tipo	Cómo instanciar
`Queue`	`Queue<String> q = new ArrayDeque<>();`
`Deque`	`Deque<String> d = new ArrayDeque<>();`
`ArrayDeque`	`ArrayDeque<String> a = new ArrayDeque<>();`

Salida Esperada

```
=== DUNGEON ENTRANCE ===

[Queue Mode]
Current Queue: [Warrior, Mage, Rogue]

[Deque Event]
[!] ALERT: Boss has appeared and skipped the line!
New Queue: [Boss, Warrior, Mage, Rogue]

[Peek]
Next to enter: Boss

[Wave Start]
Entering dungeon: Boss
Remaining: [Warrior, Mage, Rogue]

[Reinforcements Incoming]
Queue Updated: [Assassin, Warrior, Mage, Rogue, Healer]

[Wave Continues]
Entering dungeon: Assassin
Remaining: [Warrior, Mage, Rogue, Healer]

Entering dungeon: Warrior
Remaining: [Mage, Rogue, Healer]

Entering dungeon: Mage
Remaining: [Rogue, Healer]

Entering dungeon: Rogue
Remaining: [Healer]

Entering dungeon: Healer
Remaining: []


[Empty Test]
Trying to enter: null
Queue is empty.
```

IX. Ejercicio 4: La Pila de Hechizos (Stack)

- **Directorio:** ex4/
- **Archivo:** SpellStack.java
- **Método:**
- **Funciones Autorizadas:** java.util.Stack<String>, push(item), pop(), peek(), isEmpty().
- **Restricción:**
  - ✓ **No uses ArrayList**: Aunque podrías, el objetivo es usar los métodos específicos de una pila (push/pop).
  - ✓ **No accedas por índice**: En una pila real, solo te importa lo que está arriba del todo.

Objetivo - El último hechizo lanzado es el primero en deshacerse.

- Usa la estructura Stack para apilar hechizos.
- Implementa la función "Deshacer" (pop).

Salida Esperada:

```
=== SPELL CASTER LOG ===
Casting: Fireball
Casting: Ice Shield
Casting: Teleport

Top of the stack: Teleport
[!] Spell failed! Undoing last action...
Popped from stack: Teleport

Current active spells: [Fireball, Ice Shield]
```

X. Ejercicio 5: El Alquimista de Datos

- **Directorio:** ex5/
- **Archivo:** DataAlchemist.java
- **Funciones Autorizadas:** HashMap<Integer, Adventurer>, HashSet<Adventurer>, record Adventurer, Bucle for-each sobre map.values() o map.entrySet().
- **Restricción:**
- ✓ No uses arrays simples para la base de datos; la búsqueda por ID debe ser mediante Map.
- ✓ **No dupliques datos:** Si un héroe ya está en el Set de élite, no debería poder entrar dos veces.

Objetivo - Unir todo

- Crea un sistema donde tengas un Map<Integer, Player> (ID de jugador -> Registro).
- Usa Streams (opcional pero recomendado) o un bucle para filtrar solo a los jugadores de nivel > 50 y guardarlos en un Set.

Salida Esperada:

```
=== DATA ALCHEMIST LAB ===
[System] Database populated with 5 adventurers.

--- Elite Heroes (Level > 50) ---
* ID 101: Artorias (Level 85)
* ID 105: Gandalf (Level 99)

Total Elite: 2 heroes.
Transmutation complete. System: STABLE.
```

Para el filtrado, puedes usar el estilo tradicional con un bucle for o, si quieres lucirte, usa **Streams** (Java 8+):

.values().stream().filter(h -> h.level() > 50).collect(Collectors.toSet());

(Pero solo si te sientes cómodo con ello, si no, un bucle for es más que suficiente).

XI. Ejercicio 6: Misión de Comandos (ArrayList & String Processing)

- **Directorio:** ex6/
- **Archivo:** CommandMission.java
- **Funciones Autorizadas:** java.util.List, java.util.ArrayList, String.split(","). String.trim(), add(elemento), add(0, elemento), clear(), split.
- **Restricción**
  - ✓ ***No uses Arrays fijos []:*** Como no sabemos cuántos comandos enviará el usuario, es obligatorio usar ArrayList.
  - ✓ ***No ignores los espacios:*** Si el usuario escribe " move ", el sistema debe guardarlo como "move".

Objetivo

En la Dimensión Pixel, los aventureros te envían una cadena de texto con órdenes. Tu sistema debe procesar esa cadena y convertirla en una lista de misiones activa.

- Recibe una línea de comandos separada por comas (ej: explore,heal,loot,rest).
- Limpia los espacios en blanco de cada comando.
- Si el comando es abort, limpia toda la lista.
- Si el comando empieza por urgent:, debe añadirse al principio de la lista (usa un ArrayList o LinkedList).

Salida Esperada:

```
=== COMMAND MISSION PROCESSOR ===
Enter raw commands: move, urgent:save_villager, loot,
urgent:shield

[System] Processing...
Current Mission Queue: [shield, save_villager, move, loot]

--- New Entry ---
Enter raw commands: explore, abort, rest
[System] Abort command detected. Clearing list...
Current Mission Queue: [rest]
```

Para el comando **urgent:**, puedes usar el método **String.replace("urgent:", "")** o **String.substring(7)** para extraer solo el nombre de la misión antes de insertarla al principio de la lista.

XII. Ejercicio 7: El Escáner de Terreno (Arrays Estáticos)

- **Directorio:** ex7/
- **Archivo:** TerrainScanner.java
- **Funciones Autorizadas:** array[index], array.length, bucles for o while, y System.out.println().
- **Restricción:**
  - ✓ ***No redimensionar:*** Un array de 10 debe morir siendo de 10.
  - ✓ ***Cuidado con los índices:*** Recuerda que, en un array de 10, el último índice es 9. Si intentas acceder al 10, el sistema colapsará (***ArrayIndexOutOfBoundsException***).
  - ✓ No puedes usar ArrayList, Collections.reverse(), ni Arrays.sort().

Objetivo
Tu nave está escaneando la superficie de un planeta pixelado. El escáner devuelve una fila de 10 alturas de terreno (números enteros). Debes procesar estos datos "en crudo".

1. Crea un array de enteros de tamaño 10 llamado readings.
2. Llena el array con valores que tú elijas (ej: 0, 5, 2, 8, 10, 1, 0, 4, 3, 2).
3. Calcula el punto más alto (pico) y el punto más bajo (valle).
4. Calcula el promedio de altura del terreno.
5. Bonus de Alquimia: Invierte el array (que el último sea el primero) sin usar un segundo array (in-place).

```
=== TERRAIN SCANNER ===
Raw Data: [0, 5, 2, 8, 10, 1, 0, 4, 3, 2]

Analysis:
- Highest Peak: 10
- Lowest Valley: 0
- Average Altitude: 3.5

[System] Reversing terrain data for landing...
New Scan: [2, 3, 4, 0, 1, 10, 8, 2, 5, 0]
```

XIII. Ejecución

El flujo real en tu terminal

Entra en la carpeta:

Primero tienes que estar donde está el archivo. Si no, el comando javac te dirá "***archivo no encontrado***" y te entrarán ganas de tirar el PC por la ventana.

- ✓ cd Desktop/proyectos_java/ex00
- ✓ ***javac*** FtHelloForest.java **-> La Compilación (El traductor)**
- ✓ ***java*** FtHelloForest.java **-> La Ejecución (El lanzamiento)**

Herramienta	Tipo de Magia	Cuándo usarla
record	Inmutabilidad	Para datos que no deben cambiar (IDs, nombres).
ArrayList	Versatilidad	Para listas donde el orden de llegada importa.
HashSet	Velocidad	Para saber si algo existe o no, sin duplicados.
HashMap	Omnisciencia	Para encontrar un valor instantáneamente usando una clave.
Deque	Fluidez	Cuando necesitas añadir/quitar por ambos lados (colas/pilas).
[] (Array)	Estabilidad	Cuando el tamaño es sagrado y la memoria es poca.